LEZIONE - 4
TUTORAGGIO PSS

JML

Noemi Ossola

noemiossola@studenti.unimi.it

- Consente di definire contratti (precondizioni, postcondizioni e invarianti) che descrivono come un programma dovrebbe comportarsi, rendendo più semplice verificare la correttezza del codice.
  - Precondizioni → //@requires <espressione booleana>;
  - Postcondizioni → //@ensures <espressione booleana>;
  - Invarianti di classe → //@ public invariant <espressione booleana>;
- Le specifiche JML si scrivono come commenti speciali nel codice Java, in modo che non interferiscano con la compilazione:
  - //@
  - /*@ … @*/

Espressioni boolene ed operatori

- Costanti booleane → true, false
- Negazione → !
- Congiunzione → &&
- Disgiunzione → ||
- Implicazione → ==>
- Equivalenza → <==>
- Or esclusivo → <=!=>
- Operatori di confronto → ==, !=, >, <, >=, <=
- Specifica il valore restituito dal metodo → \result
- Ritornare il valore della variabile prima della chiamata del metodo → \old()

⚠️

“non”	cioè	**non A**	e si scrive	**¬A**
“e”	“	**A e B** (sia A che B)	“	**A∧B**
“o” (alternativo)	“	**A o B** (almeno uno dei due)	“	**A∨B**
“o” (disgiuntivo)	“	**A o B** (uno ed uno solo dei due)	“	**A⊕B**
“se…allora…”	“	**se A allora B**	“	**A→B**
“…se e solo se…”	“	**se A allora B e se B allora A**	“	**A↔B**

$A \iff B \equiv (A \implies B) \wedge (B \implies A)$

$\sim\exists x\, P(x) \equiv \forall x \sim P(x)$

Sintassi → (<Quantificatore> <Tipo> <Valore>; <range predicate; <espressione>)

Quantificatori:

- Universale → \forall
- Esistenziale → \exists
- Numerico → \num_of
- Generali → \sum, \product, \min, \max

- Precondizioni → Specifiche che descrivono le condizioni che devono essere vere prima che un metodo venga eseguito.
- Postcondizioni → Specifiche che descrivono le condizioni che devono essere vere alla fine dell'esecuzione di un metodo.
- Invariante → Proprietà che deve essere sempre vera durante l'intero ciclo di vita di un oggetto, ogni volta che non si è in un metodo.

Esercizio 1 – Produttore e consumatore

Scrivere una classe ProdCons che simuli il problema del produttore-consumatore.
La classe deve disporre di un buffer che contenga gli elementi prodotti dal produttore.
Il buffer deve essere modellato con un array di interi:

- Una cella vale 0 se non contiene il prodotto.
- Una cella vale 1 se lo contiene.
- Un opportuno indice deve indicare la prima cella senza prodotti.

La classe deve disporre di un `metodo produce` per produrre un prodotto ed un `metodo consume` per consumare un prodotto.
Il `costruttore` della classe deve inizializzare il buffer alla dimensione comunicata tramite parametro.
All'inizio il buffer non contiene prodotti.

- `Metodo produce` → Il metodo aggiunge un prodotto al buffer solo se c'è spazio.
  - Deve ritornare un booleano: true o false a seconda che sia stato inserito o meno il prodotto.
- `Metodo consume` → Il metodo elimina un prodotto dal buffer se questo <u>contiene almeno un prodotto</u>.
  - Deve ritornare un booleano: true o false a seconda che sia stato eliminato o meno un prodotto.

Scrivere in JML la seguente precondizione al `costruttore`:

- La capacità con cui deve essere inizializzato il buffer deve essere compresa tra 1 e 100.

Scrivere in JML le seguenti postcondizioni al `metodo produce`:

- Il buffer contiene solo 0 o 1.
- Nel buffer non c'è mai un 1 dopo uno 0.
- Il numero di 1 nel buffer dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di 1 presenti nel buffer prima dell'esecuzione del metodo.

Costruttore

Il costruttore della classe deve inizializzare il buffer alla dimensione comunicata tramite parametro. All'inizio il buffer non contiene prodotti.

```
public ProdCons(int capacity) {
        buffer = new int[capacity];
        free = 0;
    }
```

Scrivere in JML la seguente precondizione al costruttore:

- La capacità con cui deve essere inizializzato il buffer deve essere compresa tra 1 e 100

Costruttore

Scrivere in JML la seguente precondizione:

- La capacità con cui deve essere inizializzato il buffer deve essere compresa tra 1 e 100

```
//@ requires capacity >= 1 && capacity <= 100;

public ProdCons(int capacity) {
        buffer = new int[capacity];
        free = 0;
    }
```

Metodo produce()

Il metodo produce aggiunge un prodotto al buffer solo se c'è spazio. Deve ritornare un booleano: true o false a seconda che sia stato inserito o meno il prodotto.

```
public boolean produce() {
        if(free < buffer.length)
        {
                buffer[free] = 1;
                free++;
                }

                return true;
        }
        else {
                return false;
        }
}
```

Scrivere in JML le seguenti postcondizioni al metodo produce:

- Il buffer contiene solo 0 o 1.
- Nel buffer non c'è mai un 1 dopo uno 0.
- Il numero di 1 nel buffer dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di 1 presenti nel buffer prima dell'esecuzione del metodo.

produce()

Scrivere in JML le seguenti postcondizioni:

- Il buffer contiene solo 0 o 1.
- Nel buffer non c'è mai un 1 dopo uno 0.
- Il numero di 1 nel buffer dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di 1 presenti nel buffer prima dell'esecuzione del metodo.

```
//@ ensures (\forall int i; i>=0 && i<buffer.length; buffer[i]==0 || buffer[i]==1);

//@ ensures !(\exists int i; i>=0 && i<buffer.length-1; buffer[i]==0 && buffer[i+1]==1);

/*@ ensures ((\num_of int i; i>=0 && i<buffer.length; buffer[i]==1)-
   @ (\num_of int i; i>=0 && i<buffer.length; \old(buffer[i])==1)<=1); @*/

public boolean produce() {
        if(free < buffer.length)
        {
                buffer[free] = 1;
                free++;
                }

                return true;
        }
        else {
                return false;
        }
    }
```

Esercizio 2 – Natural Set

Scrivere una classe NaturalSet che rappresenti un insieme di numeri naturali.
La classe rappresenta l'insieme usando un array di interi: le celle libere dell'array contengono il valore -1.

Il costruttore della classe deve inizializzare l'array usando come dimensione il valore che viene passato come parametro al costruttore.

- All'inizio l'insieme è vuoto.

Metodo remove → La classe deve disporre di un metodo remove(int value) che permetta di rimuovere un elemento dall'insieme.

- Se il valore value è presente nell'insieme, il metodo lo rimuove e lo ritorna, altrimenti viene ritornato il valore -1.

Metodo add → La classe deve disporre di un metodo booleano add(int value) che permetta di aggiungere un elemento all'insieme.

- Se il valore value non è già presente nell'insieme, deve essere inserito e il metodo deve ritornare true.
- Se il valore è già presente nell'insieme, non deve essere inserito e il metodo deve ritornare false.

Il metodo, invece, non deve controllare che il valore che viene inserito sia positivo.

Scrivere in JML la seguente precondizione al `costruttore`:

- La **dimensione massima** dell'insieme deve essere **compresa tra 1 e 100**.

Scrivere in JML la seguente postcondizione al `costruttore` :

- L'insieme **non contiene elementi**.

Scrivere in JML la seguente precondizione al `metodo add(int value)`:

- Il **valore value è positivo**.

Scrivere in JML le seguenti postcondizione al `metodo add(int value)`:

- Nell'insieme **non esistono due elementi uguali.**
- I **numero di elementi dell'insieme** dopo l'esecuzione del metodo è **uguale o al massimo maggiore di un'unità** al numero di elementi dell'**insieme prima dell'esecuzione del metodo**.

Costruttore

La classe rappresenta l'insieme usando un **array di interi**: le **celle libere** dell'array contengono il valore -1.
Il `costruttore` della classe deve **inizializzare l'array** usando come dimensione il valore che viene passato come parametro al costruttore.
All'**inizio** l'insieme è **vuoto**.

```
public NaturalSet(int size) {
        set = new int[size];
        for(int i = 0; i < size; i++)
                set[i] = -1;
}
```

Scrivere in JML la seguente precondizione al `costruttore`:

- La **dimensione massima** dell'insieme deve essere **compresa tra 1 e 100**.

Scrivere in JML la seguente postcondizione al `costruttore` :

- L'insieme **non contiene elementi**.

Costruttore

La classe rappresenta l'insieme usando un **array di interi**: le **celle libere** dell'array contengono il valore -1.
Il costruttore della classe deve **inizializzare l'array usando come dimensione il valore che viene passato come parametro al costruttore**.
All'**inizio** l'insieme è **vuoto**.

```
//@ requires size >= 1 && size <= 100;
//@ ensures (\forall int i; i >= 0 && i < size; set[i] == -1);
public NaturalSet(int size) {
            set = new int[size];
            for(int i = 0; i < size; i++)
                  set[i] = -1;
      }
```

Scrivere in JML la seguente precondizione al costruttore:

- La dimensione massima dell'insieme deve essere compresa tra 1 e 100.

Scrivere in JML la seguente postcondizione al costruttore :

- L'insieme non contiene elementi.

Metodo add(int value)

La classe deve disporre di un metodo booleano add(int value) che permetta di aggiungere un elemento all'insieme.
Se il valore value non è già presente nell'insieme, deve essere inserito e il metodo deve ritornare true.
Se il valore è già presente nell'insieme, non deve essere inserito e il metodo deve ritornare false.

```
public boolean add(int value) {
        for(int i = 0; i < set.length; i++) {
                if(set[i] == value) {
                        return false;
                }
                else if(index == -1 && set[i] == -1) {
                        index = i;
                }
        }
        if(index == -1) {
                return false;
        }
        else {
                set[index] = value;
                return true;
        }
}
```

Scrivere in JML la seguente precondizione:

- Il valore value è positivo.

Scrivere in JML le seguenti postcondizione:

- Nell'insieme non esistono due elementi uguali.
- I numero di elementi dell'insieme dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di elementi dell'insieme prima dell'esecuzione del metodo.

Metodo add(int value)

Scrivere in JML la seguente precondizione:

- Il valore value è positivo.

Scrivere in JML le seguenti postcondizioni:

- Nell'insieme non esistono due elementi uguali.
- I numero di elementi dell'insieme dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di elementi dell'insieme prima dell'esecuzione del metodo.

```
//@ requires value >= 0;

/*@ ensures !(\exists int i; i >=0 && i < set.length;
  @ (\exists int j; j >=0 && j < set.length; ((j!=i) ==> set[i] == set[j]) && set[i] != -1));
@*/

/*@ ensures ((\num_of int i; i >= 0 && i < set.length; set[i] > -1) -
  @ (\num_of int i; i >= 0 && i < set.length; \old(set[i] > -1))) == 1
  @ ||
  @ ((\num_of int i; i >= 0 && i < set.length; set[i] > -1) -
  @ (\num_of int i; i >= 0 && i < set.length; \old(set[i] > -1))) == 0;
@*/

/*@ ensures ((\num_of int i; i >= 0 && i < set.length; set[i] > -1) -
  @ (\num_of int i; i >= 0 && i < set.length; \old(set[i] > -1))) <= 1;
@*/

public boolean add(int value) { … }
```

Esercizio 3 – Tombola

Scrivere una classe Tombola che **modelli le azioni di un croupier** nel gioco della tombola. La classe deve modellare con un **array unidimensionale di interi** (tabellone) il tabellone del croupier (il tabellone contiene 90 numeri).
Il croupier può **aggiungere delle pedine sui numeri**; nell'array, il **valore di ogni cella di indice i** indica il **numero di pedine posizionate sul numero i + 1**.

Il costruttore della classe deve **inizializzare il tabellone**, facendo in modo che **tutte le celle non contengano pedine**.

Metodo addPedina → La classe deve disporre di un metodo addPedina(int numero) che permetta di **aggiungere una pedina sul tabellone**. Il metodo **controlla che il numero inserito** sia un numero corretto (compreso tra 1 e 90) e che la cella **non contenga già una pedina**.

Metodo isCorrect → La classe deve anche disporre di un metodo booleano isCorrect(int[] numeroCartella, boolean[] numeroPresente) che prenda in input un **array di interi numeroCartella**, che contiene i numeri stampati su una cartella, e un **array di booleani numeroPresente**, che riporta quali numeri sono stati selezionati dal giocatore sulla cartella.
Il metodo deve **controllare che l'utente abbia marcato correttamente i numeri usciti sul tabellone**, cioè che **abbia segnato quelli che sono usciti e che non abbia segnato quelli che non sono usciti**. Se la cartella è stata **marcata in modo corretto** il metodo ritorna **true**, altrimenti ritorna false.

Scrivere in JML le seguenti postcondizioni al metodo `addPedina(int numero)`:

- **Non esiste** una cella che contiene **più di una pedina**.
- Il **prodotto** di tutte le celle è **minore o uguale a 1**.

Scrivere in JML le seguente precondizioni al metodo `isCorrect(int[] numeroCartella, boolean[] numeroPresente)`:

- L'array **numeroCartella** e l'array **numeroPresente** sono entrambi **lunghi 5 posizioni**.
- Tutti i numeri contenuti in **numeroCartella** sono compresi nell'intervallo **[1,90]**.

Metodo addPedina()

Il metodo addPedina(int numero) permette di aggiungere una pedina sul tabellone. Il metodo controlla che il numero inserito sia un numero corretto (compreso tra 1 e 90) e che la cella non contenga già una pedina.

```
public void addPedina(int numero) {
      if (numero >= 1 && numero <= 90 && tabellone[numero - 1] == 0) {
            tabellone[numero - 1] = tabellone[numero - 1] + 1;
      }
}
```

Scrivere in JML le seguenti postcondizioni al metodo addPedina(int numero):

- Non esiste una cella che contiene più di una pedina.
- Il prodotto di tutte le celle è minore o uguale a 1.

Metodo addPedina()

Scrivere in JML le seguenti postcondizioni al metodo :

- Non esiste una cella che contiene più di una pedina.
- Il prodotto di tutte le celle è minore o uguale a 1.

```
//@ ensures !(\exists int i; i >= 0 && i < 90; tabellone[i] > 1);

//@ ensures (\product int i; i >= 0 && i < 90; tabellone[i]) <= 1;

public void addPedina(int numero) {
        if (numero >= 1 && numero <= 90 && tabellone[numero - 1] == 0) {
                tabellone[numero - 1] = tabellone[numero - 1] + 1;
        }
}
```

Metodo isCorrect()

Il metodo booleano isCorrect(int[] numeroCartella, boolean[] numeroPresente) prende in input un **array di interi numeroCartella**, che contiene i numeri stampati su una cartella, e un **array di booleani numeroPresente**, che riporta quali numeri sono stati selezionati dal giocatore sulla cartella. Il metodo deve **controllare che l'utente abbia marcato correttamente i numeri usciti sul tabellone.** Se la cartella è stata **marcata in modo corretto** il metodo ritorna **true**, altrimenti ritorna **false**.

```
private boolean isCorrect(int[] numeroCartella, boolean[] numeroPresente) {
        for(int i = 0; i < numeroCartella.length; i++) {
                int index = numeroCartella[i] - 1;
                if((numeroPresente[i] && tabellone[index] == 0) ||
                   (!numeroPresente[i] && tabellone[index] > 0)) {
                       return false;
                }
        }
        return true;
}
```

Scrivere in JML le seguente precondizioni al metodo isCorrect(int[] numeroCartella, boolean[] numeroPresente):

- L'array **numeroCartella** e l'array **numeroPresente** sono entrambi **lunghi 5 posizioni**.
- Tutti i numeri contenuti in **numeroCartella** sono compresi nell'intervallo **[1,90]**.

Metodo isCorrect()

Scrivere in JML le seguente precondizioni al metodo isCorrect(int[] numeroCartella, boolean[] numeroPresente):

- L'array numeroCartella e l'array numeroPresente sono entrambi lunghi 5 posizioni.
- Tutti i numeri contenuti in numeroCartella sono compresi nell'intervallo [1,90].

```
//@ requires numeroCartella.length == 5 && numeroPresente.length == 5;

//@ requires (\forall int i; i >= 0 && i < 5; numeroCartella[i] > 0 && numeroCartella[i] <= 90);

private boolean isCorrect(int[] numeroCartella, boolean[] numeroPresente) {
            for(int i = 0; i < numeroCartella.length; i++) {
                  int index = numeroCartella[i] - 1;
                  if((numeroPresente[i] && tabellone[index] == 0) ||
                     (!numeroPresente[i] && tabellone[index] > 0)) {
                        return false;
                  }
            }
            return true;
}
```

Esercizio 4 – Sudoku

1	2	0	0
0	0	1	0
2	0	4	0
0	0	0	3

Table 11: Sudoku non risolto

1	2	3	4
3	4	1	2
2	3	4	1
4	1	2	3

Table 12: Sudoku risolto

Scrivere una `classe Sudoku` che permetta di giocare ad una **versione semplificata del sudoku** in cui la scacchiera è di dimensione 4×4.
Una **cella** si considera **vuota** se contiene il numero **0**. Il gioco è risolto se:

- Ogni **riga** contiene **tutti i numeri dell'intervallo [1, 4] senza ripetizioni.**
- Ogni **colonna** contiene **tutti i numeri dell'intervallo [1, 4] senza ripetizioni.**
- Le **quattro sotto-scacchiere 2×2** contengono **tutti i numeri dell'intervallo [1, 4] senza ripetizioni.**

Il `costruttore` della classe deve **inizializzare la scacchiera in modo che il sudoku sia vuoto.**

`Metodo add` → La classe deve disporre di un `metodo booleano add(int r, int c, int value)` che permetta di **aggiungere un numero nella scacchiera**; il metodo ritorna **true** se l'**inserimento viene eseguito**, **false** altrimenti.

`Metodo isMaybeSolved` → La classe deve anche disporre di un `metodo booleano isMaybeSolved` che dica se la scacchiera **soddisfa una condizione necessaria** (ma non sufficiente) affinchè possa essere considerata risolta:

- Ogni **riga** deve **sommare a 10.**
- Ogni **colonna** deve **sommare a 10.**
- Le **quattro sotto-scacchiere 2×2** devono **sommare tutte a 10.**

Scrivere in JML la seguente precondizione al metodo `add(int r, int c, int value)`:

- Tutti gli elementi sulla riga r sono diversi da value.

Scrivere in JML le seguenti postcondizioni al metodo `add(int r, int c, int value)`:

- Il numero di celle vuote è minore od uguale a 16.
- Il metodo ha ritornato true se e solo se la decisione della regola if contenuta nel metodo valuta a true.

Metodo add()

La classe deve disporre di un metodo booleano add(int r, int c, int value) che permetta di **aggiungere un numero nella scacchiera**; il metodo ritorna **true** se l'**inserimento viene eseguito**, **false** altrimenti.

```
public boolean add(int r, int c, int value) {
            if(r >= 0 && r <= 3 && c >= 0 && c <= 3 && value >= 1 && value <= 4) {
                  board[r][c] = value;
                  return true;
            } else
                  return false;
      }
```

Scrivere in JML la seguente precondizione al metodo add(int r, int c, int value):

- **Tutti gli elementi** sulla **riga r** sono **diversi da value**.

Scrivere in JML le seguenti postcondizioni al metodo add(int r, int c, int value):

- Il **numero di celle vuote** è **minore od uguale a 16.**
- Il metodo ha ritornato **true se e solo se** la decisione della regola **if** contenuta nel metodo valuta a **true**.

Metodo add()

Scrivere in JML la seguente precondizione:

- Tutti gli elementi sulla riga r sono diversi da value.

Scrivere in JML le seguenti postcondizioni:

- Il numero di celle vuote è minore od uguale a 16.
- Il metodo ha ritornato true se e solo se la decisione della regola if contenuta nel metodo valuta a true.

```
//@ requires (\forall int j; j >= 0 && j <= 3; board[r][j] != value);
//@ ensures (\sum int i; i >= 0 && i <= 3; (\num_of int j; j >= 0 && j <= 3; board[i][j] == 0)) <= 16;
//@ ensures \result <==> (r >= 0 && r <= 3 && c >= 0 && c <= 3 && value >= 1 && value <= 4);

public boolean add(int r, int c, int value) {
        if(r >= 0 && r <= 3 && c >= 0 && c <= 3 && value >= 1 && value <= 4) {
                board[r][c] = value;
                return true;
        } else
                return false;
    }
```

Esercizio 5 – Ostello

Scrivere una `classe Ostello` per modellare un ostello.
L'ostello ha **50 letti in totale**, distribuiti in **10 stanze da 5 letti ciascuna**: ogni letto può essere **libero** o **occupato**.
Lo **stato dei letti** nelle stanze deve essere modellato con una **matrice di booleani** (le **righe** sono le **stanze** e le **colonne** i **letti**). All'inizio **tutti i letti sono liberi**.

`Metodo checkin` → La classe dispone di un `metodo checkin(int stanza, int letto)` che permette di **occupare un determinato letto** in una determinata stanza.

- Il metodo **controlla che i valori letto e stanza siano corretti** e che il **letto selezionato sia libero**.
- Il metodo ritorna **true** se il **letto viene occupato**, false altrimenti.

`Metodo ottimizzaOstello` → L'ostello vuole **minimizzare il numero di stanze usate**. A tal fine la classe deve disporre di un `metodo void ottimizzaOstello()` che **sposti tutti i clienti nelle prime stanze libere** (a partire dalla prima stanza e dal primo letto).

- Per esempio, se c'è un solo cliente per ogni stanza (quindi in totale ci sono 10 clienti), il metodo sposta tutti i clienti nelle prime due stanze.

Scrivere in JML la seguente postcondizione al `costruttore`:

- All'inizio tutti i letti sono liberi.

Scrivere in JML le seguenti postcondizioni al metodo `checkin(int stanza, int letto)`:

- Tutti i letti diversi dal letto indentificato dai parametri stanza e letto non hanno cambiato il loro stato.
- Il numero di letti occupati è minore o uguale a 50.
- Se stanza è negativo, il metodo ha ritornato false.

Costruttore

L'ostello ha **50 letti in totale**, distribuiti in **10 stanze da 5 letti ciascuna**: ogni letto può essere **libero** o **occupato**.
Lo **stato dei letti** nelle stanze deve essere modellato con una **matrice di booleani** (le **righe** sono le **stanze** e le **colonne** i **letti**). All'inizio **tutti i letti sono liberi**.

```
public Ostello() {
        lettoLibero = new boolean[10][5];
                for(int r = 0; r < 10; r++) {
                        for(int c = 0; c < 5; c++) {
                                lettoLibero[r][c] = true;
                }
        }
```

Scrivere in JML la seguente postcondizione al `costruttore`:

- **All'inizio** tutti i **letti** sono **liberi**.

Costruttore

Scrivere in JML la seguente postcondizione:

- All'inizio tutti i letti sono liberi.

```
/*@ ensures (\forall int i; i >= 0 && i < 10;
@ (\forall int j; j >= 0 && j < 5; lettoLibero[i][j]));
/*@

      public Ostello() {
            lettoLibero = new boolean[10][5];
                  for(int r = 0; r < 10; r++) {
                        for(int c = 0; c < 5; c++) {
                              lettoLibero[r][c] = true;
                  }
            }
```

Metodo checkin()

Il metodo `checkin(int stanza, int letto)` permette di occupare un determinato letto in una determinata stanza.

- Il metodo controlla che i valori letto e stanza siano corretti e che il letto selezionato sia libero.
- Il metodo ritorna true se il letto viene occupato, false altrimenti.

```
public boolean checkin(int stanza, int letto) {
        if(stanza >= 0 & stanza < 10 & letto >= 0 & letto < 5) {
                if(lettoLibero[stanza][letto]) {
                        lettoLibero[stanza][letto] = false;
                        return true;
                }
        }
        return false;
}
```

Scrivere in JML le seguenti postcondizioni al metodo `checkin(int stanza, int letto)`:

- Tutti i letti diversi dal letto indentificato dai parametri stanza e letto non hanno cambiato il loro stato.
- Il numero di letti occupati è minore o uguale a 50.
- Se stanza è negativo, il metodo ha ritornato false.

Metodo checkin()

Scrivere in JML le seguenti postcondizioni:

- Tutti i letti diversi dal letto indentificato dai parametri stanza e letto non hanno cambiato il loro stato.
- Il numero di letti occupati è minore o uguale a 50.
- Se stanza è negativo (allora) il metodo ha ritornato false.

```
/*@ ensures (\forall int i; i >= 0 && i < 10;
  @  (\forall int j; j >= 0 && j < 5; (i != stanza || j != letto) ==> \old(lettoLibero[i][j]) == lettoLibero[i][j]));
@*/

//@ ensures (\sum int i; i >= 0 && i < 10; (\num_of int j; j >= 0 && j < 5; !lettoLibero[i][j])) <= 50;

//@ ensures stanza < 0 ==> !\result;

public boolean checkin(int stanza, int letto) {
        if(stanza >= 0 & stanza < 10 & letto >= 0 & letto < 5) {
                if(lettoLibero[stanza][letto]) {
                        lettoLibero[stanza][letto] = false;
                        return true;
                }
        }
        return false;
}
```

⚠ A ==> B

Esercizio 6 – Fila

Scrivere una `classe Fila` per modellare una fila ad uno sportello.
Ogni utente ritira un numero da 0 a 199 per effettuare un'operazione allo sportello.
Ogni operazione corrisponde a un numero stimato di minuti (da 1 a 15).
Gli operatori allo sportello chiamano il prossimo numero, in ordine crescente a partire da 0 fino a 199.
Il primo utente della giornata prende il numero 0.
Ogni nuovo utente prende il numero successivo all'ultimo ritirato.
Al massimo sono serviti 200 utenti.
La fila deve essere rappresentata da un array (fila[]) da 0 a 199 dove ogni cella di indice i corrisponde al numero di minuti stimati (intero da 1 a 15) per l'operazione dell'utente che ha preso il numero i.
Due variabili, che vanno da 0 a 199, contengono il numero del primo posto della fila libero (prossimoInFila) e il prossimo utente che sarà servito (prossimoServito).
Per esempio, se la fila è composta da cinque utenti (da 0 a 4) e l'utente 1 è allo sportello si ha che prossimoServito è 2 e prossimoInFila è 5.

Il `costruttore` della classe deve creare e inizializzare una fila vuota (con minuti tutti a 0) e le due variabili prossimoServito e prossimoInFila in modo coerente.

Metodo ritiroNumero → La classe dispone di un metodo ritiroNumero(int minuti) che restituisce il numero di fila all'utente e inserisce nella fila i minuti stimati per l'operazione.
Nel caso siano già stati inseriti in fila 200 utenti il metodo restituisce -1.

Metodo chiamataAlloSportello → La classe deve disporre di un metodo chiamataAlloSportello() che ritorni un intero (ovvero il prossimo numero da servire e che quindi viene chiamato allo sportello) e che aggiorni di conseguenza prossimoServito.
Nel caso in cui la fila sia finita (ovvero non ci sono utenti in attesa o stati stati serviti tutti e 200 gli utenti previsti) il metodo deve restituire -1.

Metodo stimaMinuti → La classe deve disporre di un metodo stimaMinuti(int numero) che, dato un numero in fila, ritorni un intero che contiene la stima del numero di minuti che deve aspettare l'utente prima che tocchi a lui (senza considerata l'utente allo sportello).
Per esempio se prossimoServito è 3 (ovvero 2 è attualmente allo sportello) e il numero dell'utente è 5 con una fila dove: i minuti dell'utente 2 sono stimati a 5, quelli di 3 sono 10, quelli di 4 sono 2, e quelli di 5 sono 3; il metodo deve restituire 12.
Nel caso in cui numero corrisponde ad un utente non in fila (ovvero un numero già chiamato o che non è stato ancora ritirato) il metodo deve restituire -1.

Scrivere in JML il seguente invariante:

- prossimoInFila ha un valore tra 0 e 200 inclusi e prossimoServito non è mai strettamente superiore a prossimoInFila.

Scrivere in JML la seguente postcondizione al costruttore:

- Non ci sono utenti in fila, verificando esclusivamente il contenuto dell'array fila[].

Scrivere in JML la seguente precondizione al metodo ritiroNumero(int minuti):

- I minuti di attesa di un utente sono compresi tra 1 e 15 inclusi.

Scrivere in JML le seguenti postcondizioni al metodo ritiroNumero(int minuti):

- Tutti gli elementi dell'array fila[] di indice strettamente inferiore a prossimoInFila hanno valore strettamente maggiori di 0.
- Almeno un utente è stato inserito in fila, verificando esclusivamente il contenuto dell'array fila[].
- Nell'array fila[] non c'è mai uno valore a 0 prima di uno strettamente maggiore di 0.

Scrivere in JML la seguente postcondizione al metodo stimaMinuti(int numero):

- Se numero è già stato chiamato il metodo restituisce -1.

Invariante

Scrivere in JML il seguente invariante:

- prossimoInFila ha un valore tra 0 e 200 inclusi e prossimoServito non è mai strettamente superiore a prossimoInFila.

```
public class Fila{
      private /*@ spec_public @*/ int[] fila;
      public int prossimoInFila;
      private /*@ spec_public @*/ int prossimoServito;

      …

}
```

Invariante

Scrivere in JML il seguente invariante:

- prossimoInFila ha un valore tra 0 e 200 inclusi e prossimoServito non è mai strettamente superiore a prossimoInFila.

```
public class Fila{
        private /*@ spec_public @*/ int[] fila;
        public int prossimoInFila;
        private /*@ spec_public @*/ int prossimoServito;

        /*@ public invariant prossimoInFila >=0 && prossimoInFila <=200
        @ && !(prossimoServito>prossimoInFila);
        @*/
…

}
```

Costruttore

Il costruttore della classe deve creare e inizializzare una fila vuota (con minuti tutti a 0) e le due variabili prossimoServito e prossimoInFila in modo coerente.

```
public Fila() {
      fila = new int[200];
      prossimoInFila = 0;
      prossimoServito = 0;
}
```

Scrivere in JML la seguente postcondizione al costruttore:

- Non ci sono utenti in fila, verificando esclusivamente il contenuto dell'array fila[].

Costruttore

Scrivere in JML la seguente postcondizione al costruttore:

- Non ci sono utenti in fila, verificando esclusivamente il contenuto dell'array fila[].

```
//@ ensures (\forall int i; i >= 0 && i < 200; fila[i] == 0);

public Fila() {
        fila = new int[200];
        prossimoInFila = 0;
        prossimoServito = 0;
}
```

ritiroNumeri()

La classe dispone di un metodo ritiroNumero(int minuti) che restituisce il numero di fila all'utente e inserisce nella fila i minuti stimati per l'operazione.
Nel caso siano già stati inseriti in fila 200 utenti il metodo restituisce -1.

```
public int ritiroNumero(int minuti) {
        int numero = -1;
        if(prossimoInFila<200 && prossimoInFila>=0) {
                fila[prossimoInFila] = minuti;
                numero = prossimoInFila++;
        }
        return numero;
}
```

Scrivere in JML la seguente precondizione al metodo ritiroNumero(int minuti):

- I minuti di attesa di un utente sono compresi tra 1 e 15 inclusi.

Scrivere in JML le seguenti postcondizioni al metodo ritiroNumero(int minuti):

- Tutti gli elementi dell'array fila[] di indice strettamente inferiore a prossimoInFila hanno valore strettamente maggiori di 0.
- Almeno un utente è stato inserito in fila, verificando esclusivamente il contenuto dell'array fila[].
- Nell'array fila[] non c'è mai uno valore a 0 prima di uno strettamente maggiore di 0.

ritiroNumeri()

Scrivere in JML la seguente precondizione:

- I minuti di attesa di un utente sono compresi tra 1 e 15 inclusi.

Scrivere in JML le seguenti postcondizioni:

- Tutti gli elementi dell'array fila[] di indice strettamente inferiore a prossimoInFila hanno valore strettamente maggiori di 0.
- Almeno un utente è stato inserito in fila, verificando esclusivamente il contenuto dell'array fila[].
- Nell'array fila[] non c'è mai uno valore a 0 prima di uno strettamente maggiore di 0.

```
//@ requires minuti >= 1 && minuti <=15;
//@ ensures (\forall int i; i >= 0 && i < prossimoInFila; fila[i]>0);
//@ ensures (\exists int i; i >= 0 && i < 200; fila[i] > 0);
//@ ensures (\forall int i; i >= 0 && i < 199; ((fila[i] == 0) ==> (fila[i+1] == 0)));
//@ ensures (\forall int i; i >= 0 && i < 199; !((fila[i] == 0) && (fila[i+1] > 0)));

public int ritiroNumero(int minuti) {
        int numero = -1;
        if(prossimoInFila<200 && prossimoInFila>=0) {
                fila[prossimoInFila] = minuti;
                numero = prossimoInFila++;
        }
        return numero;
}
```

stimaMinuti()

La classe deve disporre di un metodo stimaMinuti(int numero) che, dato un numero in fila, ritorni un intero che contiene la stima del numero di minuti che deve aspettare l'utente prima che tocchi a lui (senza considerata l'utente allo sportello).
Per esempio se prossimoServito è 3 e il numero dell'utente è 5 con una fila dove: i minuti dell'utente 2 sono stimati a 5, quelli di 3 sono 10, quelli di 4 sono 2, e quelli di 5 sono 3; il metodo deve restituire 12.
Nel caso in cui numero corrisponde ad un utente non in fila (ovvero un numero già chiamato o che non è stato ancora ritirato) il metodo deve restituire -1.

```
public int stimaMinuti(int numero){
        int risultato = 0;
        if(prossimoServito > numero || numero >= prossimoInFila)
                risultato = -1;
        else{
                for(int i=prossimoServito; i < numero; i++)
                        risultato += fila[i];
        }
        return risultato;
}
```

Scrivere in JML la seguente postcondizione al metodo stimaMinuti(int numero):

- Se numero è già stato chiamato il metodo restituisce -1.

stimaMinuti()

Scrivere in JML la seguente postcondizione:

- Se numero è già stato chiamato il metodo restituisce -1.

```
//@ ensures (prossimoServito > numero) ==> \result == -1;

public int stimaMinuti(int numero){
        int risultato = 0;
        if(prossimoServito > numero || numero >= prossimoInFila)
                risultato = -1;
        else{
                for(int i=prossimoServito; i < numero; i++)
                        risultato += fila[i];
        }
        return risultato;
}
```

Esercizio 7 – Undirected graph

Scrivere una `classe UndirectedGraph` che rappresenti un **grafo non orientato**.
La classe deve **memorizzare gli archi usando una matrice booleana** di incidenza di dimensione **n×n** (dove n è il numero di nodi): la **cella (i, j)** della matrice vale **true** se c'è un **arco dal nodo i al nodo j** (poichè l'arco non è orientato, anche (j, i) deve valere true).

Il `costruttore` della classe deve **inizializzare la matrice di incidenza** facendo in modo che questa rappresenti un **grafo di n nodi senza archi**: il valore n deve essere passato come parametro al costruttore.

`Metodo addArc` → La classe deve disporre di un `metodo addArc(int nodeA, int nodeB)` che permetta di **aggiungere archi al grafo**.

`Metodo cycle3` → La classe deve anche disporre di un `metodo booleano cycle3` che dica se il **grafo contiene almeno un** ciclo di lunghezza 3.

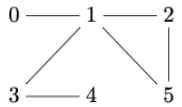

Figure 5: Grafo non orientato con ciclo di lunghezza 3 tra i nodi 1, 2 e 5

Scrivere in JML la seguente precondizione al costruttore:

- Il numero di nodi del grafo deve essere compreso tra 1 e 100.

Scrivere in JML la seguente postcondizione al costruttore:

- Il grafo non contiene archi.

Scrivere in JML la seguente precondizione al metodo addArc(int nodeA, int nodeB):

- Non esiste un arco tra nodeA e nodeB.

Scrivere in JML la seguente postcondizione al metodo addArc(int nodeA, int nodeB):

- Il numero di archi del grafo dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di archi del grafo prima dell'esecuzione del metodo (attenzione che ogni arco viene rappresentato due volte).

Costruttore

Il costruttore della classe deve inizializzare la matrice di incidenza facendo in modo che questa rappresenti un grafo di n nodi senza archi: il valore n deve essere passato come parametro al costruttore.

```
public UndirectedGraph(int numberNodes) {
        incidenceMatrix = new boolean[numberNodes][numberNodes];
}
```

Scrivere in JML la seguente precondizione al costruttore:

- Il numero di nodi del grafo deve essere compreso tra 1 e 100.

Scrivere in JML la seguente postcondizione al costruttore:

- Il grafo non contiene archi.

Costruttore

Scrivere in JML la seguente precondizione:

- Il numero di nodi del grafo deve essere compreso tra 1 e 100.

Scrivere in JML la seguente postcondizione :

- Il grafo non contiene archi.

```
//@ requires numberNodes > 0 && numberNodes <= 100;

/*@ ensures (\forall int i; i >= 0 && i < incidenceMatrix.length;
@ (\forall int j; j >= 0 && j < incidenceMatrix.length; !incidenceMatrix[i][j]));
@*/

public UndirectedGraph(int numberNodes) {
            incidenceMatrix = new boolean[numberNodes][numberNodes];
}
```

addArc()

La classe deve disporre di un metodo addArc(int nodeA, int nodeB) che permetta di aggiungere archi al grafo.

```
public boolean addArc(int nodeA, int nodeB) {
        if (nodeA >= 0 && nodeA < incidenceMatrix . length && nodeB >= 0
                && nodeB < incidenceMatrix . length ) {
                        incidenceMatrix[nodeA][nodeB] = true;
                        incidenceMatrix[nodeB][nodeA] = true;
                        return true;
        } else {
        return false;
        }
}
```

Scrivere in JML la seguente precondizione al metodo addArc(int nodeA, int nodeB):

- Non esiste un arco tra nodeA e nodeB.

Scrivere in JML la seguente postcondizione al metodo addArc(int nodeA, int nodeB):

- Il numero di archi del grafo dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di archi del grafo prima dell'esecuzione del metodo (attenzione che ogni arco viene rappresentato due volte).

addArc()

Scrivere in JML la seguente precondizione:

- Non esiste un arco tra nodeA e nodeB.

Scrivere in JML la seguente postcondizione:

- Il numero di archi del grafo dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di archi del grafo prima dell'esecuzione del metodo

```
//@ requires !incidenceMatrix[nodeA][nodeB] && !incidenceMatrix[nodeB][nodeA];

/*@ ensures ((\sum int i; i >= 0 && i < incidenceMatrix.length;
@ (\num_of int j; j >= 0 && j < incidenceMatrix.length; incidenceMatrix[i][j])) -
@ (\sum int i; i >= 0 && i < incidenceMatrix.length;
@ (\num_of int j; j >= 0 && j < incidenceMatrix.length; \old(incidenceMatrix[i][j])))) <= 2;
@*/

/*@ ensures ((\sum int i; i >= 0 && i < incidenceMatrix.length;
@ (\num_of int j; j >= 0 && j < incidenceMatrix.length; incidenceMatrix[i][j])) -
@ (\sum int i; i >= 0 && i < incidenceMatrix.length;
@ (\num_of int j; j >= 0 && j < incidenceMatrix.length; \old(incidenceMatrix[i][j])))) == 2
@ ||
@ ((\sum int i; i >= 0 && i < incidenceMatrix.length;
@ (\num_of int j; j >= 0 && j < incidenceMatrix.length; incidenceMatrix[i][j])) -
@ (\sum int i; i >= 0 && i < incidenceMatrix.length;
@ (\num_of int j; j >= 0 && j < incidenceMatrix.length; \old(incidenceMatrix[i][j])))) == 0;
@*/

public boolean addArc(int nodeA, int nodeB) {…}
```

Esercizio 8 – Ora di punta

Scrivere una classe OraDiPunta che modelli una **versione semplificata del gioco Ora di punta**.
Su una **griglia 6×6** sono disposte **6 macchine, numerate con i numeri da 1 a 6** (ogni macchina ha dimensione 1×1); **celle vuote** sono **indicate con 0**.
Obiettivo del gioco è **portare la macchina numero 1** (chiamata macchina rossa) all'uscita della griglia, che corrisponde alla **cella con** indici (2, 5) (le coordinate iniziano da 0).
Il costruttore della classe deve disporre le macchine sulla griglia come mostrato in Tabella:

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	2
2	0	0	1	0	0	**3**
3	0	0	0	0	0	4
4	0	5	0	0	0	0
5	0	0	6	0	0	0

`Metodo spostaMacchina` → La classe deve disporre di un `metodo booleano spostaMacchina(int riga, int colonna, int dir)` che permetta di **muovere una macchina sulla griglia**; il metodo ritorna **true** se una **macchina è stata spostata**, false altrimenti.
I **parametri riga e colonna** indicano una **cella** della griglia; **dir** indica una **direzione** nel seguente modo:

- 1: verso l'alto;
- 2: verso destra;
- 3: verso il basso;
- 4: verso sinistra.

Il metodo dove **controllare che i valori passati siano corretti**: riga e colonna sono indici corretti della griglia e dir è una delle quattro direzioni.
Se in **riga e colonna non c'è una macchina**, **niente deve accadere**.
Se una **macchina non si può spostare in direzione** dir perchè ostruita da un'altra macchina o dai limiti della griglia, non deve essere spostata.

`Metodo macchinaRossaUscita` → La classe dispone di un `metodo booleano macchinaRossaUscita` che **dice se la macchina rossa ha raggiunto l'uscita.**

Scrivere in JML la seguente postcondizione al costruttore:

- Nella griglia ci sono 6 celle non vuote.

Scrivere in JML la seguente precondizione al metodo spostaMacchina(int riga, int colonna, int dir):

- I parametri attuali identificano una cella della griglia e una delle quattro direzioni.

Scrivere in JML le seguenti postcondizioni al metodo spostaMacchina(int riga, int colonna, int dir):

- Il prodotto di tutte le celle è 0;
- Non esiste una cella con valore minore di 0.

Costruttore

Il costruttore della classe deve disporre le macchine sulla griglia come mostrato in Tabella:

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	2
2	0	0	1	0	0	**3**
3	0	0	0	0	0	4
4	0	5	0	0	0	0
5	0	0	6	0	0	0

```
public OraDiPunta() {

        griglia = new int[6][6];
        griglia[2][2] = 1;
        griglia[1][5] = 2;
        griglia[2][5] = 3;
        griglia[3][5] = 4;
        griglia[4][1] = 5;
        griglia[5][2] = 6;
}
```

Scrivere in JML la seguente postcondizione al costruttore:

- Nella griglia ci sono 6 celle non vuote.

Costruttore

Scrivere in JML la seguente postcondizione :

- Nella griglia ci sono 6 celle non vuote.

```
/*@ ensures (\sum int i; i >= 0 && i < 6;
@ (\num_of int j; j >= 0 && j < 6; griglia[i][j] != 0)) == 6;
@*/

public OraDiPunta() {

            griglia = new int[6][6];
            griglia[2][2] = 1;
            griglia[1][5] = 2;
            griglia[2][5] = 3;
            griglia[3][5] = 4;
            griglia[4][1] = 5;
            griglia[5][2] = 6;
}
```

spostaMacchina

Metodo spostaMacchina → La classe deve disporre di un metodo booleano spostaMacchina(int riga, int colonna, int dir) che permetta di muovere una macchina sulla griglia; il metodo ritorna true se una macchina è stata spostata, false altrimenti.
Il metodo dove controllare che i valori passati siano corretti.
Se in riga e colonna non c'è una macchina, niente deve accadere.
Se una macchina non si può spostare in direzione dir perchè ostruita da un'altra macchina o dai limiti della griglia, non deve essere spostata.

```
public boolean spostaMacchina(int riga, int colonna, int dir) {
        if(riga >= 0 && riga < 6 && colonna >= 0 && colonna < 6) {
                int car = griglia[riga][colonna];
                switch(dir) {
                        case 1: {…}
                        …
                        default
                }
        }
}
```

Scrivere in JML la seguente precondizione al metodo spostaMacchina(int riga, int colonna, int dir):

- I parametri attuali identificano una cella della griglia e una delle quattro direzioni.

Scrivere in JML le seguenti postcondizioni al metodo spostaMacchina(int riga, int colonna, int dir):

- Il prodotto di tutte le celle è 0;
- Non esiste una cella con valore minore di 0.

spostaMacchina()

Scrivere in JML la seguente precondizione al metodo:

- I parametri attuali identificano una cella della griglia e una delle quattro direzioni.

Scrivere in JML le seguenti postcondizioni al metodo:

- Il prodotto di tutte le celle è 0;
- Non esiste una cella con valore minore di 0.

```
//@ requires riga >= 0 && riga < 6 && colonna >= 0 && colonna < 6 && dir >=1 && dir <=4;

//@ ensures (\product int i; i >= 0 && i < 6; (\product int j; j >= 0 && j < 6; griglia[i][j])) == 0;

//@ ensures !(\exists int i; i >= 0 && i < 6; (\exists int j; j >= 0 && j < 6; griglia[i][j] < 0));

public boolean spostaMacchina(int riga, int colonna, int dir) {
        if(riga >= 0 && riga < 6 && colonna >= 0 && colonna < 6) {
                int car = griglia[riga][colonna];
                        switch(dir) {
                                case 1: {…}
                                …
                                default
                        }
        }
}
```

Esercizio 9 – Rubik2D

Scrivere una classe Rubik2D per rappresentare una versione semplificata del cubo di Rubik in cui solo una faccia del cubo viene rappresentata. All'inizio, la configurazione della faccia è:

1	0	1
2	1	2
0	2	0

Metodo gira → La classe dispone di un metodo booleano gira() che, dati un indice di colonna idCol, sposta verso l'alto la colonna (in modo ciclico).

- Il metodo ritorna true se idCol indica una colonna valida e quindi lo spostamento viene fatto, false altrimenti.

Metodo isSolved → La classe deve disporre di un metodo booleano isSolved() che dica se la faccia è risolta, cioè se su ogni riga c'è lo stesso numero.

2	2	2
0	0	0
1	1	1

Scrivere in JML la seguente postcondizione al `costruttore`

- Il valore di tutte le celle della faccia sono comprese tra 0 e 2.

Scrivere in JML le seguenti postcondizioni al metodo `gira(int idCol)`:

- Ci sono tre celle con valore uguale a 0.
- La somma di tutte le celle è 9.
- Le celle che non appartengono alla colonna id Col non hanno cambiato valore rispetto a prima dell'esecuzione del metodo.

All'inizio, la configurazione della faccia è:

1	0	1
2	1	2
0	2	0

Costruttore

```
public Rubik2D() {
     faccia = new int[3][3];
     faccia[0][0] = 1;
     faccia[0][2] = 1;
     faccia[1][1] = 1;
     faccia[1][0] = 2;
     faccia[1][2] = 2;
     faccia[2][1] = 2;
}
```

Scrivere in JML la seguente postcondizione al `costruttore`

- Il valore di tutte le celle della faccia sono comprese tra 0 e 2.

Costruttore

Scrivere in JML la seguente postcondizione:

- Il valore di tutte le celle della faccia sono comprese tra 0 e 2.

```
/*@ ensures (\forall int i; i >= 0 && i < 3;
@(\forall int j; j >= 0 && j < 3; faccia[i][j] >= 0 && faccia[i][j] < 3));
*/@

    public Rubik2D() {
        faccia = new int[3][3];
        faccia[0][0] = 1;
        faccia[0][2] = 1;
        faccia[1][1] = 1;
        faccia[1][0] = 2;
        faccia[1][2] = 2;
        faccia[2][1] = 2;
    }
```

Gira(int idCol)

La classe dispone di un metodo booleano gira() che, dati un indice di colonna idCol, sposta verso l'alto la colonna (in modo ciclico).
Il metodo ritorna true se idCol indica una colonna valida e quindi lo spostamento viene fatto, false altrimenti.

```
public boolean gira(int idCol) {
        if(idCol >= 0 && idCol < 3) {
                int primo = faccia[0][idCol];
                for(int i = 0; i < 2; i++) {
                        faccia[i][idCol] = faccia[i + 1][idCol];
                }
                faccia[2][idCol] = primo;
                return true;
        }
        return false;
}
```

Scrivere in JML le seguenti postcondizioni al metodo gira(int idCol):

- Ci sono tre celle con valore uguale a 0.
- La somma di tutte le celle è 9.
- Le celle che non appartengono alla colonna id Col non hanno cambiato valore rispetto a prima dell'esecuzione del metodo.

Gira(int idCol)

Scrivere in JML le seguenti postcondizioni:

- Ci sono tre celle con valore uguale a 0.
- La somma di tutte le celle è 9.
- Le celle che non appartengono alla colonna id Col non hanno cambiato valore rispetto a prima dell'esecuzione del metodo.

```
//@ ensures (\sum int i; i >= 0 && i < 3; (\num_of int j; j >= 0 && j < 3; faccia[i][j] == 0)) == 3;
//@ ensures (\sum int i; i >= 0 && i < 3; (\sum int j; j >= 0 && j < 3; faccia[i][j])) == 9;
/*@ ensures (\forall int i; i >= 0 && i < 3;
@ (\forall int j; j >= 0 && j < 3; j != idCol ==> faccia[i][j] == \old(faccia[i][j])));
@*/

public boolean gira(int idCol) {
        if(idCol >= 0 && idCol < 3) {
                int primo = faccia[0][idCol];
                for(int i = 0; i < 2; i++) {
                        faccia[i][idCol] = faccia[i + 1][idCol];
                }
                faccia[2][idCol] = primo;
                return true;
        }
        return false;
}
```